

78. Subsets Medium

Acceptance: 81.9% | [LeetCode](#)

Given an integer array `nums` of **unique** elements, return *all possible subsets (the power set)*.

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

```
**Input:** nums = [1,2,3]
**Output:** [[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]
```

Example 2:

```
**Input:** nums = [0]
**Output:** [[], [0]]
```

Constraints:

- `1 <= nums.length <= 10`
- `-10 <= nums[i] <= 10`
- All the numbers of `nums` are **unique**.

Claude Opus Solution

Subsets — Power Set Generation

1. Intuition

The **key insight** is that for each element in the array, we have a binary choice: **include it** or **exclude it** in a subset. With `n` elements, this gives us exactly 2^n subsets (the power set).

There are two natural mental models: - **Decision tree (backtracking):** At each index, branch into "take this element" vs "don't take it." - **Subset extension:** Start with `[[] ]`. For every new number, append it to copies of all existing subsets, doubling the count.

The "unique elements" guarantee means we never have to worry about deduplication — every subset we generate is automatically distinct.

2. Approach (Backtracking — same family as reference)

We use DFS where `index` marks where we are allowed to start picking. By only picking elements at index `i` or later, we avoid generating the same subset in different orders (e.g., we never produce both `[1,2]` and `[2,1]`).

```
function subsets(nums):
    result = []
    dfs(start=0, current=[])
    return result

function dfs(start, current):
    # Every node in the recursion tree IS a valid subset
    record a copy of current into result

    # Try extending current with each remaining element
    for i from start to len(nums)-1:
        current.append(nums[i])    # choose
        dfs(i + 1, current)       # explore (i+1 prevents reuse/duplicates)
        current.pop()             # un-choose (backtrack)
```

3. Python Solution

Approach A: Backtracking (recommended)

```
from typing import List

class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        result = [] # holds all subsets

        def dfs(start: int, current: List[int]) -> None:
            # KEY: every recursion call corresponds to one valid subset.
            # We append a *copy* (list(current)) because `current` is mutated
            # later during backtracking - storing the reference would corrupt it.
            result.append(list(current))
            for i in range(start, len(nums)):
                current.append(nums[i])
                dfs(i + 1, current)
                current.pop()
```

```

        result.append(current[:])

        # Try adding each element from `start` onward.
        # Using `start` (not 0) ensures we only move forward, so we never
        # generate permutations like [2,1] of an already-seen subset [1,2].
        for i in range(start, len(nums)):
            current.append(nums[i])    # CHOOSE: include nums[i]
            dfs(i + 1, current)       # EXPLORE: recurse with next index
            current.pop()              # UN-CHOOSE: backtrack to try others

    dfs(0, [])
    return result

```

Approach B: Iterative Subset Extension (often cleaner)

```

from typing import List

class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        # Start with the single empty subset.
        result = [[]]

        for num in nums:
            # For each existing subset, create a new one that also includes `num`.
            # This doubles the number of subsets each iteration: 1 -> 2 -> 4 -> ...
            result += [subset + [num] for subset in result]

        return result

```

Approach C: Bit Masking (each subset ↔ a binary number)

```

from typing import List

class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        n = len(nums)
        result = []

        # There are 2^n subsets. Each integer mask in [0, 2^n) encodes one:
        # bit j set -> include nums[j].
        for mask in range(1 << n):    # 1 << n == 2^n
            subset = []
            for j in range(n):
                if mask & (1 << j):    # is the j-th bit set?
                    subset.append(nums[j])
            result.append(subset)

        return result

```

4. Complexity Analysis

Let $n = \text{len}(\text{nums})$. There are 2^n subsets, and the average subset length is $n/2$, so total output size is $O(n \cdot 2^n)$. You cannot beat this — it's the size of the answer itself.

Approach	Time	Space (excluding output)	Space (with output)
A: Backtracking	$O(n \cdot 2^n)$	$O(n)$ recursion depth + <code>current</code>	$O(n \cdot 2^n)$
B: Iterative	$O(n \cdot 2^n)$	$O(1)$ extra	$O(n \cdot 2^n)$
C: Bit Masking	$O(n \cdot 2^n)$	$O(1)$ extra	$O(n \cdot 2^n)$

Why $O(n \cdot 2^n)$ time? - We build 2^n subsets. - Copying each subset into the result costs up to $O(n)$. - Product $\rightarrow O(n \cdot 2^n)$.

Space note: The $O(n \cdot 2^n)$ is unavoidable because it's the output. Auxiliary space for backtracking is only $O(n)$ (recursion stack + the `current` list).

5. Alternative Approaches Summary

Approach	Idea	Pros	Cons
Backtracking (A)	DFS with choose/explore/un-choose	Generalizes to combinations, permutations, Subsets II	Recursion overhead; must remember to copy
Iterative extension (B)	Double the list per element	Very concise, no recursion	Slightly less intuitive as a "template"
Bit masking (C)	Each mask in $[0, 2^n)$ selects elements	Elegant, no recursion, explicit 1-to-1 mapping	Only practical for small n (here $n \leq 10$, perfect)

Recommendation: Learn **Approach A** as a reusable template — the same `choose → recurse → un-choose` skeleton solves [Subsets II](#), [Permutations](#), and [Combination Sum](#). For this specific problem with tiny n , Approach B or C is the shortest to write correctly.

Codex Solution

1. Intuition

For each element in `nums`, we have **two choices**:

1. Include it in the current subset
2. Exclude it from the current subset

Since there are n elements and each element has 2 choices, there are:

$[2^n]$

total subsets.

A natural way to generate all subsets is **backtracking**:

- Build a subset step by step.
- Add the current subset to the answer.
- Try adding each remaining number.
- Backtrack by removing the last added number.

This is a classic **DFS / backtracking** pattern.

2. Approach

Pseudocode

```
function subsets(nums):
    result = []

    function backtrack(start, current_subset):
        add a copy of current_subset to result

        for i from start to len(nums) - 1:
            add nums[i] to current_subset
            backtrack(i + 1, current_subset)
            remove nums[i] from current_subset

    backtrack(0, [])
    return result
```

Why `start` is needed

The `start` index ensures that:

- We only move forward in the array.
- We do not reuse elements.
- We avoid duplicate subsets such as `[1,2]` and `[2,1]`.

3. Python Solution

```
from typing import List

class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        # This will store all generated subsets
        result = []

        def backtrack(start: int, current: List[int]) -> None:
            """
            Generate all subsets starting from index 'start'.

            Parameters:
            start:
                The index from which we are allowed to choose the next element.

            current:
                The current subset being built.
            """

            # Add a copy of the current subset to the result.
            # We use current[:] because current will be modified later.
            result.append(current[:])

            # Try adding each remaining element to the current subset
            for i in range(start, len(nums)):
                # Choose nums[i]
                current.append(nums[i])

                # Explore further choices after index i
                backtrack(i + 1, current)

                # Undo the choice so we can try the next possibility
                current.pop()

            # Start backtracking from index 0 with an empty subset
            backtrack(0, [])

        return result
```

4. Complexity Analysis

Let:

[$n = \text{len}(\text{nums})$]

Time Complexity

[$O(n \cdot 2^n)$]

There are 2^n subsets total.

For each subset, we may copy up to n elements when doing:

```
result.append(current[:])
```

Therefore, the total time complexity is:

[$O(n \cdot 2^n)$]

Space Complexity

[$O(n \cdot 2^n)$]

The output itself contains 2^n subsets, and each subset can contain up to n elements.

So the space needed for the result is:

[$O(n \cdot 2^n)$]

The extra recursion stack and temporary `current` list use:

[$O(n)$]

So:

- Output space: $O(n * 2^n)$
- Auxiliary space excluding output: $O(n)$

5. Alternative Approaches

Alternative 1: Iterative Expansion

Start with the empty subset. For each number, add it to all existing subsets.

```
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        result = [[]]

        for num in nums:
            new_subsets = []

            for subset in result:
                new_subsets.append(subset + [num])

            result.extend(new_subsets)

        return result
```

Example

For `nums = [1,2,3]`:

```
Start: [[]]

Add 1:
[[], [1]]

Add 2:
[[], [1], [2], [1,2]]

Add 3:
[[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]
```

Time complexity:

[$O(n \cdot 2^n)$]

Space complexity:

[$O(n \cdot 2^n)$]

Alternative 2: Bitmasking

Each subset can be represented by a binary mask.

For example, with `nums = [1,2,3]`:

```
000 -> []
001 -> [1]
010 -> [2]
011 -> [1,2]
100 -> [3]
101 -> [1,3]
110 -> [2,3]
111 -> [1,2,3]
```

```
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        n = len(nums)
        result = []

        # There are 2^n possible masks
        for mask in range(1 << n):
            subset = []
```

```
for i in range(n):  
    # If the i-th bit is set, include nums[i]  
    if mask & (1 << i):  
        subset.append(nums[i])  
  
    result.append(subset)  
  
return result
```

Time complexity:

[$O(n \cdot 2^n)$]

Space complexity:

[$O(n \cdot 2^n)$]

The backtracking solution is usually preferred because it is clean, intuitive, and generalizes well to many subset and combination problems.